DSA0602 Data Handling and Visualization

Slot C

CO2 – Assessment Test c (AT2)

Data Interpretation Exercise

Name: Jai Ganesh Rao N

Reg No:192524073

Q1. Podcast Episode Length vs. Listener Retention

Objective: Generate data for 22 podcast episodes and visualize the relationship between episode length and average listener retention. Slight jitter helps distinguish overlapping points.

Python Source Code

```
# Q1: Podcast episode length versus listener retention
import pandas as pd
import matplotlib.pyplot as plt

# Load the simulated dataset.
df = pd.read_csv("../datasets/q1_podcast_retention.csv")

# Add a small, repeatable jitter to reduce point overlap.
jitter = (pd.Series(range(len(df))) % 3 - 1) * 0.25
x = df["episode_length_minutes"] + jitter
y = df["average_listener_retention_percent"] + jitter

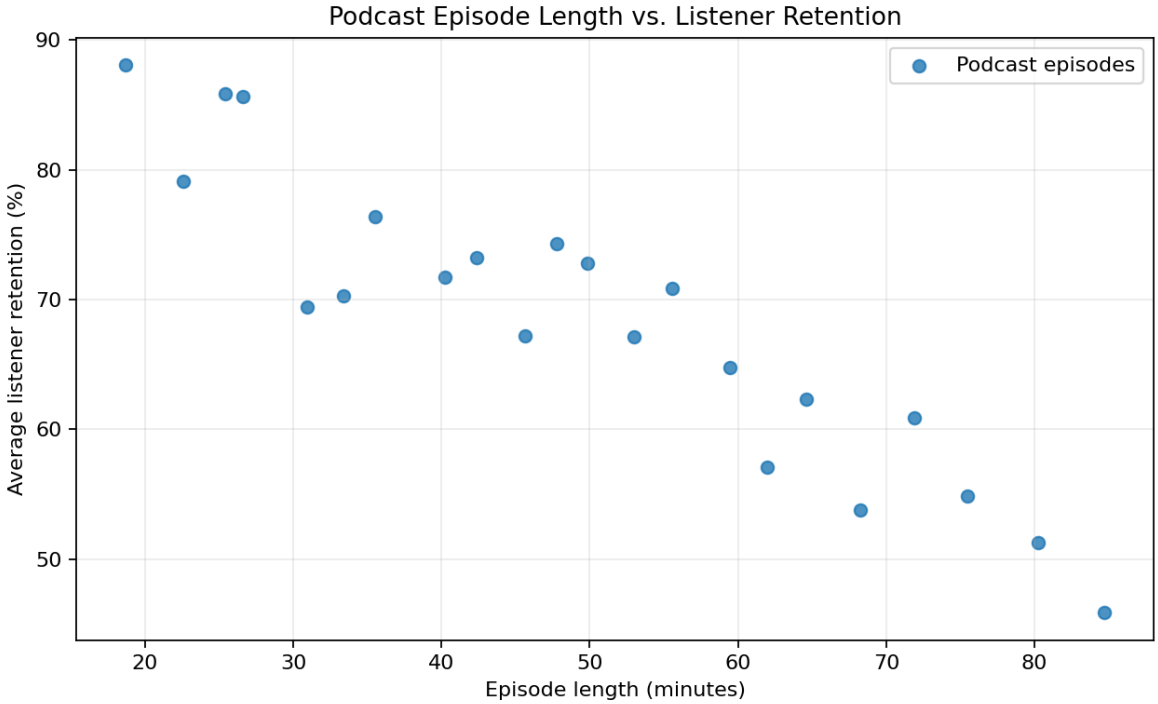
# Create and format the scatter plot.
plt.figure(figsize=(8, 5))
plt.scatter(x, y, alpha=0.8, label="Podcast episodes")
plt.title("Podcast Episode Length vs. Listener Retention")
plt.xlabel("Episode length (minutes)")
plt.ylabel("Average listener retention (%)")
plt.grid(True, alpha=0.25)
plt.legend()
plt.tight_layout()

# Save the chart as a PNG image and display it.
plt.savefig("../charts/q1_episode_retention.png", dpi=160)
plt.show()
```

Dataset file: datasets/q1_podcast_retention.csv

Output chart: charts/q1_episode_retention.png

Output Chart



Q2. Coffee Chain Outlets Across Eight Cities

Objective: Plot the locations of eight cities and use marker size to represent average daily coffee outlet footfall. This is a latitude/longitude coordinate plot, not a detailed street map.

Python Source Code

```
# Q2: Coffee outlet locations and average daily footfall
import pandas as pd
import matplotlib.pyplot as plt

# Load city coordinates and simulated footfall data.
df = pd.read_csv("../datasets/q2_coffee_outlets.csv")

# Draw city markers; larger markers indicate higher footfall.
plt.figure(figsize=(9, 6))
plt.scatter(df["longitude"], df["latitude"],
            s=df["average_daily_footfall"] / 8,
            alpha=0.65, edgecolors="black", label="Coffee outlet city")

# Add city names beside their coordinates.
for _, row in df.iterrows():
    plt.annotate(row["city"], (row["longitude"], row["latitude"]),
                 xytext=(5, 5), textcoords="offset points", fontsize=8)

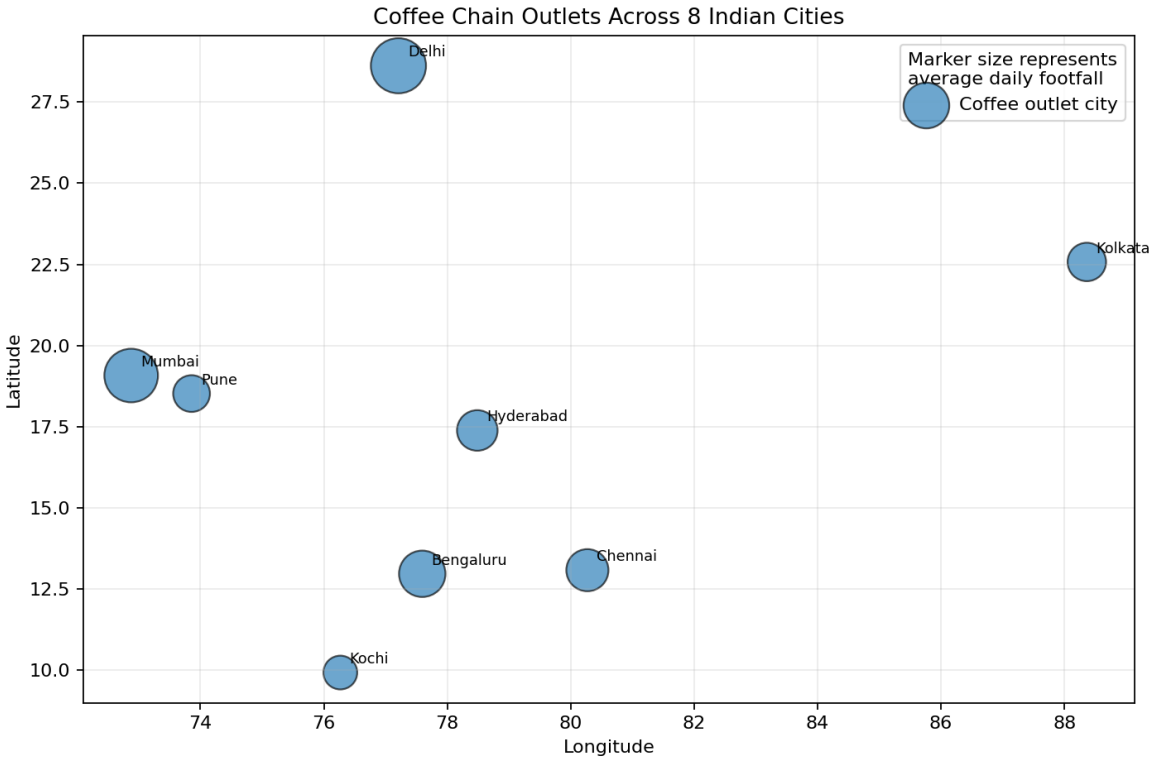
plt.title("Coffee Chain Outlets Across 8 Indian Cities")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.legend(title="Marker size represents\naverage daily footfall")
plt.grid(True, alpha=0.25)
plt.tight_layout()

# Export and display the chart.
plt.savefig("../charts/q2_coffee_city_map.png", dpi=160)
plt.show()
```

Dataset file: datasets/q2_coffee_outlets.csv

Output chart: charts/q2_coffee_city_map.png

Output Chart



Q3. Climate Science Citation Network

Objective: Build a directed network from an edge list of eight climate-science papers. The red node highlights the paper with the highest number of incoming citations.

Python Source Code

```
# Q3: Directed climate science citation network
import pandas as pd
import networkx as nx
import matplotlib.pyplot as plt

# Load paper details and citation edge list.
papers = pd.read_csv("../datasets/q3_papers.csv")
edges = pd.read_csv("../datasets/q3_citation_edge_list.csv")

# Add papers as nodes and citations as directed edges.
# Each edge points from the citing paper to the paper it cites.
G = nx.DiGraph()
G.add_nodes_from(papers["paper_id"])
G.add_edges_from(zip(edges["citing_paper_id"], edges["cited_paper_id"]))

# Count incoming citations for each paper.
incoming = dict(G.in_degree())
highest = max(incoming.values())
pos = nx.spring_layout(G, seed=17)
colors = ["tomato" if incoming[n] == highest else "skyblue" for n in G.nodes()]
sizes = [700 + incoming[n] * 250 for n in G.nodes()]

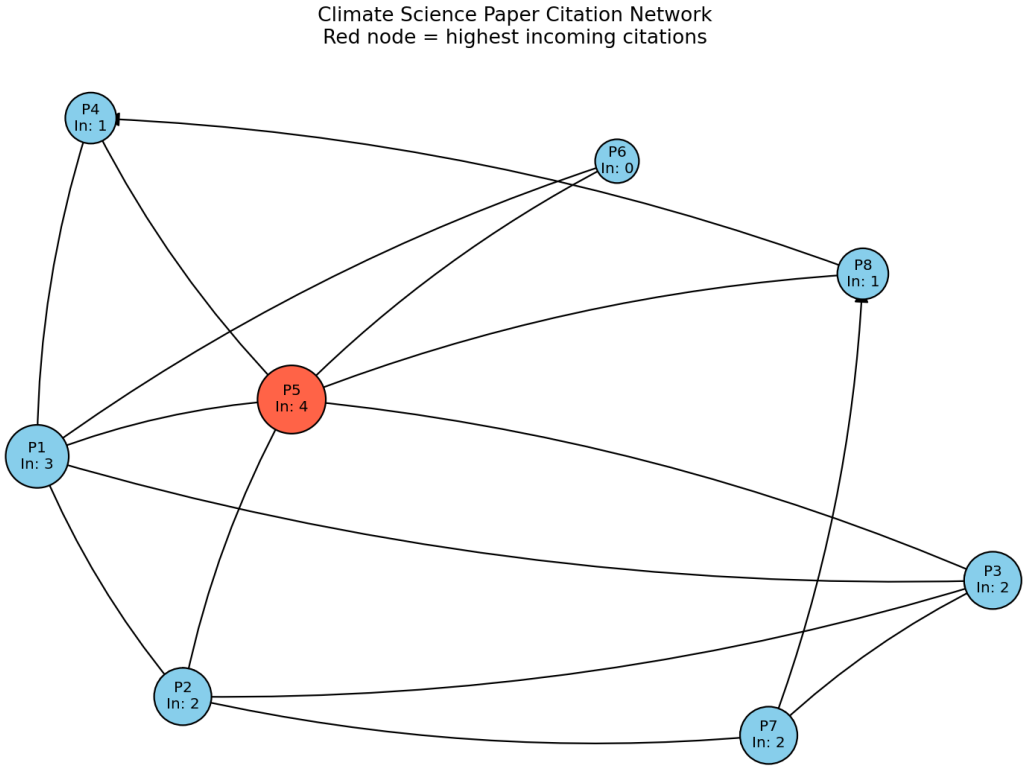
# Draw nodes, directed edges, and labels.
plt.figure(figsize=(10, 7))
nx.draw_networkx_nodes(G, pos, node_color=colors, node_size=sizes, edgecolors="black")
nx.draw_networkx_edges(G, pos, arrows=True, arrowstyle="-|>", arrowsize=18,
                       connectionstyle="arc3,rad=0.08")
labels = {n: f"{n}\nIncoming: {incoming[n]}" for n in G.nodes()}
nx.draw_networkx_labels(G, pos, labels=labels, font_size=9)
plt.title("Climate Science Paper Citation Network\nRed node = highest incoming citations")
plt.axis("off")
plt.tight_layout()

# Save and display the network graph.
plt.savefig("../charts/q3_citation_network.png", dpi=160)
plt.show()
```

Dataset file: datasets/q3_citation_edge_list.csv and datasets/q3_papers.csv

Output chart: charts/q3_citation_network.png

Output Chart



Q4. Daily Ride-Hailing Trips with Rolling Average

Objective: Generate 30 days of simulated ride-hailing trip counts and plot the daily values with a seven-day rolling average overlay.

Python Source Code

```
# Q4: Daily ride-hailing trips and seven-day rolling average
import pandas as pd
import matplotlib.pyplot as plt

# Load the daily trip dataset and parse its date column.
df = pd.read_csv("../datasets/q4_daily_ride_trips.csv", parse_dates=["date"])

# Calculate a seven-day rolling average to smooth daily variation.
df["rolling_average"] = df["daily_trip_count"].rolling(
    window=7, min_periods=1
).mean()

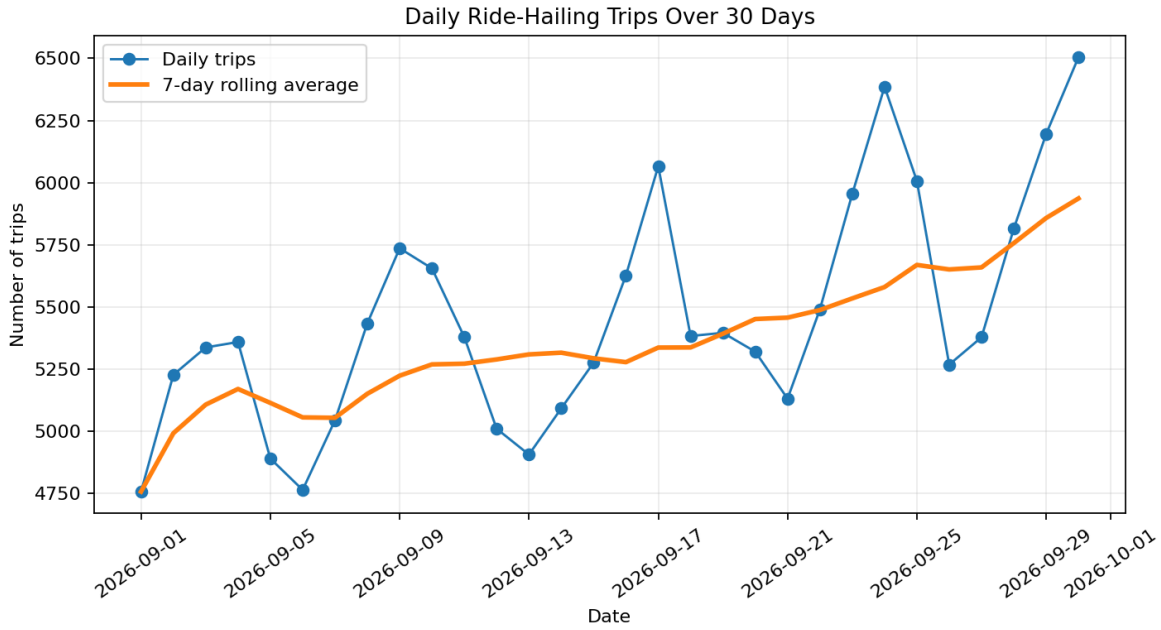
# Plot daily counts and the smoothed trend.
plt.figure(figsize=(9, 5))
plt.plot(df["date"], df["daily_trip_count"], marker="o",
         linewidth=1.3, label="Daily trips")
plt.plot(df["date"], df["rolling_average"], linewidth=2.5,
         label="7-day rolling average")
plt.title("Daily Ride-Hailing Trips Over 30 Days")
plt.xlabel("Date")
plt.ylabel("Number of trips")
plt.xticks(rotation=35)
plt.grid(True, alpha=0.25)
plt.legend()
plt.tight_layout()

# Save and display the time-series chart.
plt.savefig("../charts/q4_ride_trips_trend.png", dpi=160)
plt.show()
```

Dataset file: datasets/q4_daily_ride_trips.csv

Output chart: charts/q4_ride_trips_trend.png

Output Chart



Q5. Comparing Six Podcasts with Parallel Coordinates

Objective: Compare six podcasts across episode count, average listeners, average rating, and average episode length. Metrics are normalized to 0–1 because their original units differ.

Python Source Code

```
# Q5: Compare podcasts using a parallel-coordinates chart
import pandas as pd
import matplotlib.pyplot as plt

# Load the simulated podcast comparison dataset.
df = pd.read_csv("../datasets/q5_podcast_comparison.csv")
metrics = [
    "episode_count",
    "average_listeners_thousands",
    "average_rating",
    "average_episode_length_minutes"
]

# Normalize each metric to the range 0–1.
scaled = df.copy()
for col in metrics:
    low, high = df[col].min(), df[col].max()
    scaled[col] = (df[col] - low) / (high - low) if high != low else 0

# Plot one line for each podcast across all four metrics.
x = range(len(metrics))
plt.figure(figsize=(10, 6))
for _, row in scaled.iterrows():
    plt.plot(x, [row[c] for c in metrics], marker="o",
             linewidth=1.8, label=row["podcast"])

plt.title("Podcast Comparison: Parallel Coordinates (Normalized Metrics)")
plt.xlabel("Metric")
plt.ylabel("Normalized value (0 = lowest, 1 = highest)")
plt.xticks(list(x), [
    "Episode count", "Avg. listeners\n(thousands)",
    "Average rating", "Episode length\n(minutes)"
])
plt.ylim(-0.05, 1.05)
plt.grid(True, axis="y", alpha=0.25)
plt.legend(bbox_to_anchor=(1.02, 1), loc="upper left", fontsize=8)
plt.tight_layout()

# Export the chart and display it.
```

```
plt.savefig("../charts/q5_podcast_parallel_coordinates.png",
            dpi=160, bbox_inches="tight")
plt.show()
```

Dataset file: datasets/q5_podcast_comparison.csv

Output chart: charts/q5_podcast_parallel_coordinates.png

Output Chart

